

这份文件是 Anthropic 的一篇工程实践文章，主题是：**如何为 AI Agent 设计有效的评估体系 evals**。它不是在讲某一个具体指标，而是在讲一套从 0 到 1 建立 Agent 评估系统的方法论。核心观点是：**Agent 越强，越难评估；但越难评估，越需要评估。**

## 一句话总结

AI Agent 的 eval，本质上不是简单问“回答对不对”，而是要测试它在多轮任务、工具调用、环境变化、最终结果和用户体验上的整体表现。

通俗来说，普通大模型像“考试答题”，你问一句，它答一句；Agent 更像“实习生做项目”，它可能要查资料、调工具、改文件、调用 API、和用户来回沟通。评价它不能只看最后一句话写得好不好，而要看：**==它有没有真的把事情办成、过程有没有乱来、成本和速度是否可接受、结果是否稳定。==**

---

## 1. 为什么 Agent eval 比普通 LLM eval 更复杂？

文件开头强调，Agent 有三个特点：**自主性、智能性、灵活性**。这些能力让 Agent 有用，但也让评估更困难。

普通单轮 eval 通常是：

输入 prompt → 模型输出 response → grader 打分

但 Agent eval 往往是：

输入任务 → Agent 多轮思考和行动 → 调工具 → 修改环境状态 → 继续调整 → 最终完成任务 → grader 检查结果

比如一个 coding agent，不是只输出一段代码，而是可能要读文件、改文件、跑测试、修 bug。最终评估时，不只看它“说自己修好了”，而是要看测试是否真的通过、环境状态是否真的改变。

这里有一个很关键的概念：**outcome 比 transcript 更重要。**

通俗解释：

Agent 说“我已经帮你订好机票了”，这只是 transcript 里的话；  
数据库里真的出现了机票订单，才是 outcome。

所以评估 Agent 时，不能只看它说了什么，还要看它 **实际做成了什么**。

---

## 2. Agent eval 的基本组成

文章给出了一组非常重要的定义：

| 概念                       | 含义         | 通俗解释               |
|--------------------------|------------|--------------------|
| Task                     | 一个测试任务     | 给 Agent 的一道题       |
| Trial                    | 一次运行尝试     | 同一道题跑一次            |
| Grader                   | 打分逻辑       | 判卷老师               |
| Transcript / Trace       | 完整轨迹       | Agent 做题全过程录像      |
| Outcome                  | 最终环境状态     | 事情是否真的办成           |
| Eval harness             | 评估运行框架     | 自动出题、运行、记录、打分的系统   |
| Agent harness / Scaffold | Agent 运行框架 | 让模型能调用工具、循环执行任务的外壳 |
| Eval suite               | 一组任务集合     | 一套试卷               |

这几个概念对你后面做 Agent 项目非常关键。尤其要区分：

**Agent harness ≠ Model 本身。**

你评估的不是一个裸模型，而是：

模型 + prompt + 工具 + 工具调用逻辑 + 环境 + 记忆/状态管理 + 输出格式

这和你之前做 Market Insight Agent、Ecno-assist 的经验很相关：实际产品效果往往不是模型单点能力，而是整个系统链路的能力。

## 3. 为什么要做 eval?

文章的核心理由是：没有 eval，团队会陷入被动修 bug 的循环。

没有 eval 时，开发流程往往是：

用户说 Agent 变差了 → 团队手动复现 → 改 prompt / 改工具 → 上线 → 又引入新问题 → 继续救火

这就像没有单元测试的软件开发。你不知道一次改动到底是优化了，还是只是把问题从 A 地方挪到了 B 地方。

有 eval 之后，你至少可以做到：

- 1. 每次改 prompt、改工具、换模型前，先跑一遍测试集；

2. 判断是否出现 regression;
3. 比较不同模型或不同版本;
4. 把用户反馈转化成可重复测试的任务;
5. 让产品、工程、研究团队围绕同一套指标沟通。

通俗理解：

eval 是 Agent 产品的“体检表 + 回归测试 + 产品需求说明书”。

它不只是为了打分，更是为了定义“什么叫好”。

---

## 4. 三类 grader：代码打分、模型打分、人工打分

文章把 Agent 的 grader 分成三类。

### 1) Code-based grader：代码/规则打分（deterministic grader/确定性评测器）

包括：

- `exact match`（完全匹配） / `regex match`（正则匹配） / `fuzzy match`（模糊匹配）；
- 单元测试；
- 静态分析；
- 工具调用检查；
- 环境状态检查；
- `transcript` 分析，比如轮数、`token`、延迟。

优点是：快、便宜、客观、可复现。

缺点是：容易僵硬，不适合开放式任务。

通俗理解：

适合判断“有没有真的完成某个明确动作”，比如订单是否生成、测试是否通过、文件是否存在。

### 2) Model-based grader：LLM-as-judge

包括：

- `rubric-based scoring`：提前制定\*\*明确的打分标准、评分维度、分值档位\*\*（评分细则 / 打分表），让裁判 LLM 对照细则逐项打分；类似考试阅卷的\*\*评分标准\*\*。

- **natural language assertions**: 用\*\*自然语言写判断语句\*\*，让裁判 LLM 判断 “这句话断言是否成立”，只输出是 / 否、成立 / 不成立；相当于给裁判下达一条条判断指令
- **pairwise comparison**: 把\*\*两个模型 / 两个版本 Agent 的回答放在一起\*\*，让裁判 LLM 二选一：哪个更好、更准确、更贴合需求；不单独打分，只做横向对比。
- **reference-based evaluation**: 提供\*\*标准答案 / 参考范文（Reference）\*\*，让裁判 LLM 对比模型输出和参考答案，判断相似度、正确率、完整性；类似考试 “对照标准答案判卷”。
- **多 judge 共识**: 不止用一个 LLM 当裁判，同时调用\*\*多个独立裁判模型\*\*（可以是不同模型、不同 prompt、不同账号），综合所有人的判断结果，取多数意见作为最终结论；规避单个 LLM 判分偏心、出错的问题。

优点是：**灵活，能处理开放式输出。**

缺点是：**不稳定、成本更高，需要人工校准。**

通俗理解：

适合判断“回答质量好不好”“语气是否合适”“总结是否全面”“是否遵循指令”。

### 3) Human grader：人工评估

包括专家评审、抽样检查、A/B 测试、人工一致性校验等。

优点是：**最接近真实用户判断。**

缺点是：**慢、贵、不可频繁大规模运行。**

文章的建议很务实：

能用 deterministic grader 就尽量用；  
必须判断开放式质量时，用 LLM grader；  
人工评估主要用于校准 LLM grader，而不是每次都全量人工打分。

---

## 5. Capability eval 和 Regression eval 的区别

这是文件里非常重要的一组概念。

### Capability eval：能力评估

它问的是：

这个 Agent 现在能做到什么程度？

这种 eval 一开始不应该太容易，最好有一定失败率。因为它的作用是给团队一个“爬坡目标”。

比如：

Agent 能不能完成复杂代码修复？  
能不能做长链路研究？  
能不能在多轮对话里完成退款流程？

## Regression eval：回归评估

它问的是：

这个 Agent 以前能做的事情，现在还能不能稳定做到？

这种 eval 应该接近 100% 通过率。只要分数下降，就说明新版本可能破坏了旧能力。

通俗解释：

Capability eval 像“挑战题”，用来看上限；  
Regression eval 像“基础题”，用来看有没有退步。

一个成熟流程是：

某类 capability eval 被模型稳定攻克后，就可以“毕业”成为 regression suite。

这和软件工程里的单元测试、集成测试很像。

---

## 6. 不同类型 Agent 应该怎么评估？

文件分别讨论了几类典型 Agent。

### 1) Coding Agent

Coding Agent 最适合用 deterministic grader，因为代码可以跑测试。

常用评估方式：

- 单元测试是否通过；
- fail-to-pass / pass-to-pass；
- lint、type check、安全扫描；
- 代码质量 rubric；
- 工具调用和文件修改轨迹分析。

通俗理解：

Coding Agent 不要只看它解释得多好，关键看代码是否真的跑通、有没有破坏原功能、有没有引入安全问题。

## 2) Conversational Agent

比如客服、销售、教练类 Agent。

它的难点是：**任务完成和沟通质量都要评估。**

可能的指标包括：

- 退款是否真的处理完成；
- 是否验证身份；
- 是否在 10 轮内解决；
- 是否有同理心；
- 解释是否清楚；
- 是否基于工具结果回答，而不是瞎编。

通俗理解：

客服 Agent 不是只要“态度好”，也不是只要“把事办了”。它要既办成事，又让用户觉得过程可靠、清楚、舒服。

## 3) Research Agent

Research Agent 的评估最难，因为研究任务通常开放、主观、动态变化。

常见维度：

- groundedness：结论是否有来源支持；
- coverage：关键事实是否覆盖；
- source quality：来源是否权威；
- coherence：综合是否连贯；
- completeness：是否全面。

通俗理解：

研究 Agent 的评估不是问“标准答案是什么”，而是问“这份研究是否有证据、是否覆盖关键问题、是否没有胡说”。

这和你金融研究/战略研究场景高度相关。比如让 Agent 写一份“券商 PB 业务竞争格局分析”，不能只看文字是否通顺，而要看：

1. 是否引用了可靠数据源；

2. 是否覆盖收入结构、客户结构、费率趋势、竞争格局；
3. 是否区分事实与推断；
4. 是否避免把不确定信息写成确定结论。

## 4) Computer Use Agent

这类 Agent 通过 GUI 操作软件，比如浏览器、办公软件、企业系统。

评估方式通常是检查最终环境状态：

- 页面是否导航正确；
- 表单是否提交；
- 订单是否创建；
- 文件是否修改；
- 系统配置是否改变。

通俗理解：

不要只看它有没有点到确认页面，而要看后台状态是否真的变了。

---

## 7. $\text{pass}@k$ 和 $\text{pass}^k$ ：如何理解 Agent 的不稳定性？

文章特别强调：Agent 有随机性，同一个任务跑多次，结果可能不同。

所以不能只跑一次就下结论。

### $\text{pass}@k$

含义是：

跑  $k$  次，只要有一次成功，就算成功。

适合那些“多试几次，只要有一个可用答案就行”的场景。

比如代码生成、候选方案生成。

### $\text{pass}^k$

含义是：

跑  $k$  次，必须每次都成功。

适合那些“每个用户都希望稳定成功”的场景。

比如客服退款、机票预订、企业流程自动化。

文章举了一个直观例子：如果单次成功率是 75%，连续 3 次都成功的概率是：

$$0.75^3 \approx 42\%$$

这说明一个 Agent 看起来“单次表现还不错”，但如果要求稳定可靠，可能远远不够。

通俗解释：

pass@k 看的是“有没有一次灵光”；  
pass^k 看的是“靠不靠谱”。

对真实产品来说，很多用户面对的是 pass^k，而不是 pass@k。

---

## 8. 从 0 到 1 建 eval 的路线图

文章给出了一套非常实用的路线。

### Step 0：早点开始

不要等有几百个测试任务再开始。20-50 个来自真实失败的小任务，就足够作为第一版。

这点非常重要。很多团队会误以为 eval 是大厂才做的复杂系统，但实际上一开始可以很轻量。  
==

### Step 1：从你已经手动测试的东西开始

把你每次上线前都会手动检查的 case，转成 eval。

比如：

这个 Agent 是否会正确调用搜索工具？  
是否会在信息不足时说不知道？  
是否会生成符合 schema 的 JSON？  
是否会错误改写数据事实？

### Step 2：写清楚任务和参考答案

好的 task 应该让两个专家独立判断时，能得出同样的 pass/fail 结论。

如果任务描述含糊，模型失败可能不是能力问题，而是题目本身有问题。

通俗解释：



eval 题目不能像“请写得好一点”这么模糊，而要写清楚什么叫好、什么叫不合格。

## Step 3: 构建平衡测试集

既要测试“应该做”的情况，也要测试“不应该做”的情况。

文章举了搜索工具的例子：

- 有些问题应该搜索，比如天气、最新新闻；
- 有些问题不该搜索，比如“谁创立了 Apple”这种常识问题。

如果只测试“该搜索时有没有搜索”，模型可能学成“什么都搜索”。

通俗解释：

eval 不能只防漏做，也要防乱做。

## Step 4: 搭建稳定 eval harness

每次 trial 要有干净、隔离的环境。

否则可能出现：

- 上一次运行留下文件，影响下一次；
- 缓存让模型占便宜；
- 资源不足导致多个任务同时失败；
- 环境 bug 被误判为模型能力差。

这对 Agent 尤其重要，因为 Agent 会改变环境状态。

## Step 5: 设计 grader

文章建议：

- 尽量评估结果，而不是强行规定路径；
- 对复杂任务设置 partial credit（不用非黑即白，完成一部分任务，给一部分分数）；
- LLM judge 要和人工专家校准；
- rubric 要结构化（把评分标准写成条理清晰、层级分明、可逐条核对的正式规则表）；
- 给 judge “Unknown” 选项，避免它硬判；
- 防止 Agent 利用漏洞“作弊”。

这点特别关键：**不要过度检查工具调用顺序。**

因为 Agent 可能找到设计者没想到但有效的路径。更好的做法是：

不要只问它有没有按你想的方式做；  
要问它有没有真的把事情做成。

## Step 6：读 transcript/trace

文章非常强调：一定要读轨迹。

分数只是表象，transcript 才能告诉你：

- 是 Agent 真错了；
- 还是 grader 太僵硬；
- 是任务描述有歧义；
- 还是工具/环境出了问题。

通俗解释：

看 eval 分数像看体检指标；读 transcript 才像看病历和片子。

## Step 7：关注 eval 饱和

如果一个 eval 已经接近 100% 通过，它就不再适合作为能力提升指标，只适合作为回归测试。

否则你会低估新模型的真实进步。

## Step 8：长期维护

Eval suite 不是一次性资产，而是活文档。

产品经理、客户成功、销售、领域专家都应该能贡献 eval case。因为他们最知道真实用户期望是什么。

# 9. eval 不是唯一方法，要和其他方法组合

文章最后强调，自动化 eval 只是理解 Agent 质量的一层。

完整体系还包括：

| 方法                    | 作用                  |
|-----------------------|---------------------|
| Automated evals       | 开发阶段快速迭代，CI/CD 回归测试 |
| Production monitoring | 观察真实用户行为和线上错误       |
| A/B testing           | 验证真实业务指标变化          |

| 方法                       | 作用                |
|--------------------------|-------------------|
| User feedback            | 捕捉未预料的问题          |
| Manual transcript review | 理解微妙失败模式          |
| Systematic human studies | 校准主观任务和 LLM judge |

文章用了“瑞士奶酪模型”来解释：每一层方法都有洞，单独一层都挡不住所有问题；多层叠加，才能提高整体可靠性。

通俗理解：

eval 不是万能质检员，而是质量体系中的第一道防线。

## 10. 启发

**第一，eval 是 Agent 工程能力的核心，不是附属品。**

会写 prompt、会调 API 只是基础；真正进入工程化阶段，必须能定义任务、设计 grader、搭建 harness、分析 trace、做 regression。

**第二，Agent 产品的质量不是“感觉不错”，而是要被任务集和指标约束。**  
这和你之前提到的“从感觉不错到可度量优化”完全一致。

**第三，金融研究 Agent 最适合用混合评估。**  
比如你的 Market Insight Agent 可以这样设计：

| 评估对象        | 推荐 grader            |
|-------------|----------------------|
| 数据计算是否正确    | deterministic check  |
| 异常检测是否触发正确  | rule-based grader    |
| 是否引用了正确数据   | groundedness check   |
| 解释是否过度归因    | LLM rubric + 人工抽检    |
| 报告结构是否完整    | schema validation    |
| 是否改变事实口径    | contract-based check |
| 延迟、成本、token | transcript metric    |

**第四，eval case 最好来自真实失败。**

比如模型误判市场异动原因、遗漏关键指标、把不确定线索写成确定因果、调用搜索工具过度或不足，这些都可以沉淀成 eval tasks。

---

## 最后用一个类比总结

做 Agent eval，就像管理一个新员工。

你不能只看他说：

“我完成了。”

你要看：

1. 任务有没有真的完成；
2. 有没有遵守边界；
3. 有没有用对工具；
4. 过程有没有可追溯；
5. 成本和时间是否合理；
6. 换一天再做是否还能稳定完成；
7. 出错后你能不能定位原因。

所以这篇文章真正想表达的是：

**Agent eval 的目标，不是给模型打一个漂亮分数，而是建立一套让团队能持续改进、放心上线、快速定位问题的工程反馈系统。**